INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Title: Insecure Deserialization

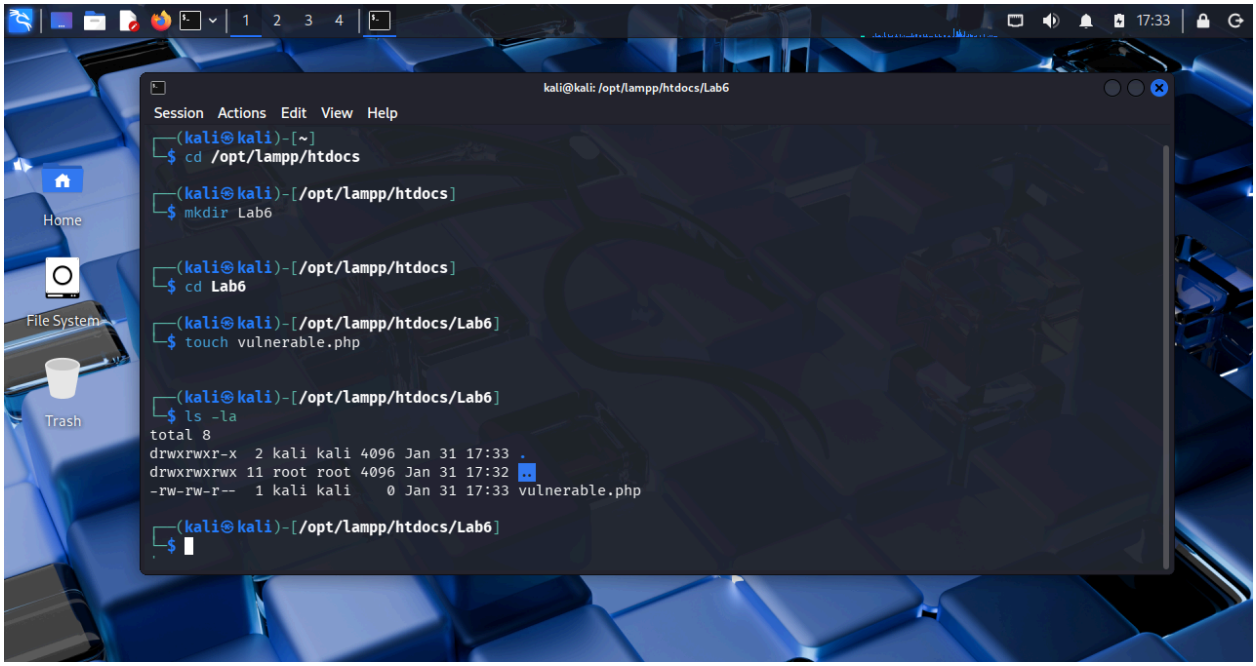
Objective

- Understand insecure deserialization.
- Exploit a vulnerable PHP application.
- Apply proper mitigation to prevent the attack.

Task 1: Creating a Vulnerable Application

A PHP file named vulnerable.php was created, inside the Lab6 folder created earlier
The application accepts serialized data from user input and directly unserializes it without validation.

This makes the application vulnerable to object injection.



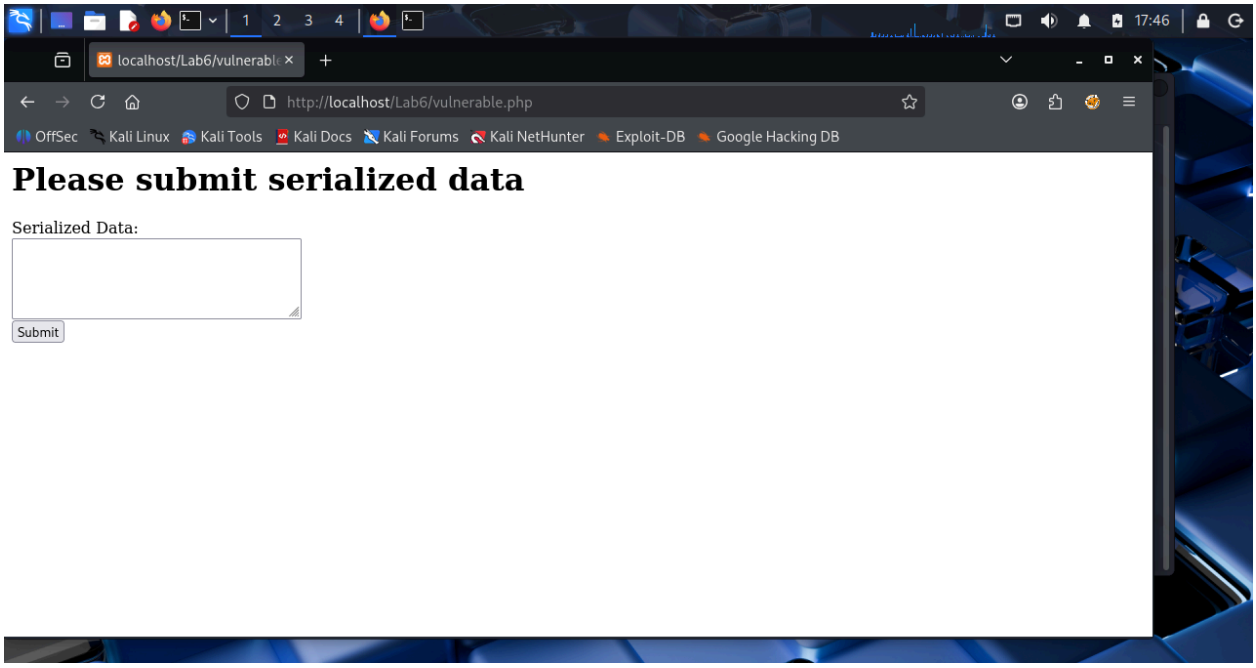
Task 2: Exploiting Insecure Deserialization

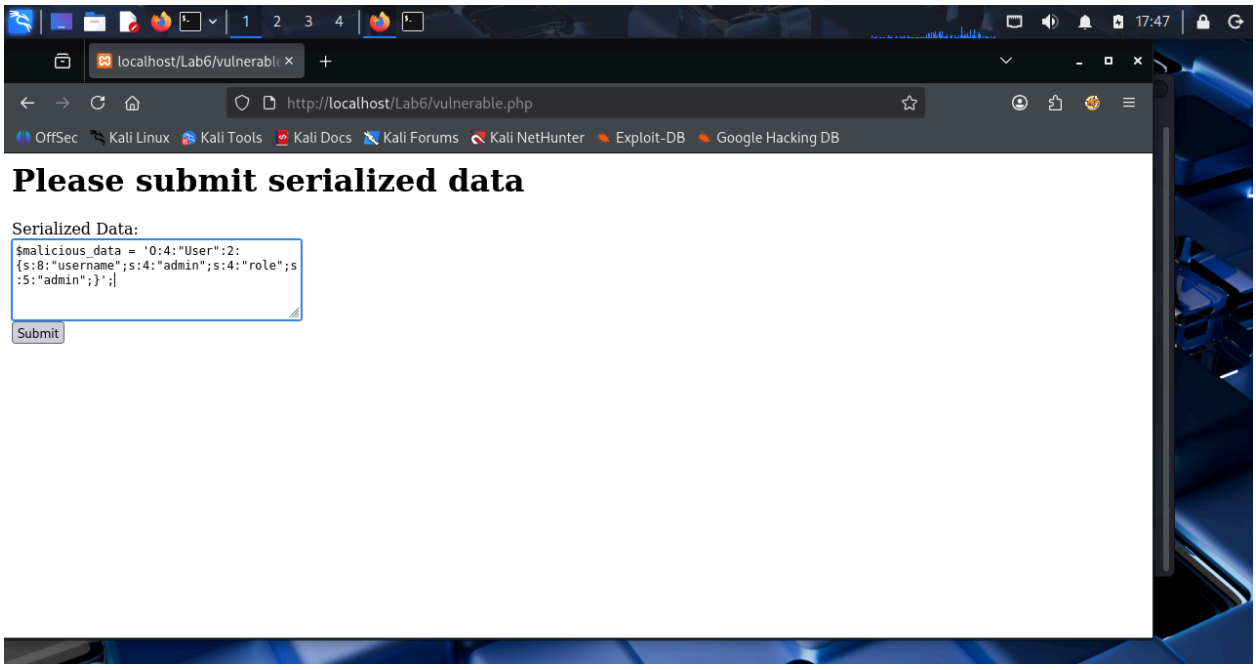
The application was accessed through the browser.
A serialized User object was manually submitted through the form.

Serialized payload used:

O:4:"User":2:{s:8:"username";s:5:"admin";s:4:"role";s:5:"admin";}

The application trusted the input and created an admin user without authentication.



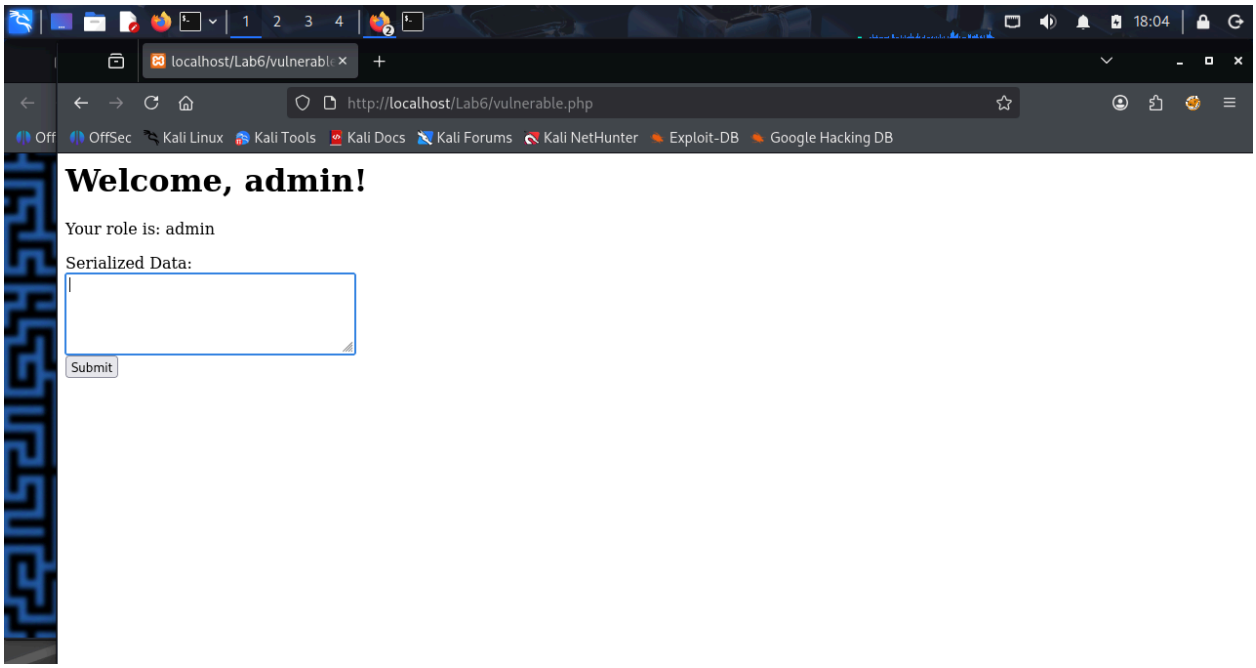


Observed Issue

Initially, the application returned a **500 Internal Server Error**.
This occurred due to improper handling of unserialized input.

Error reporting was enabled to reveal and fix the issue.
After correction, the exploit worked successfully.

This shows insecure deserialization can also cause denial of service.



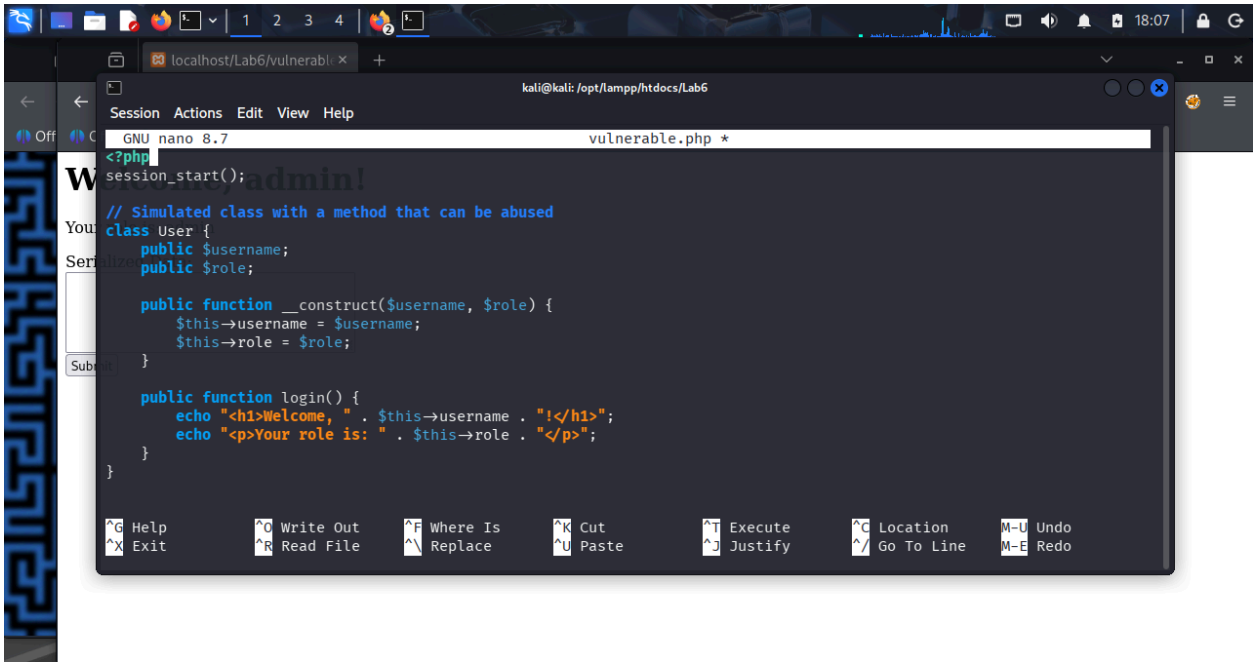
Task 3: Mitigation

The application was refactored to remove unserialize().

Changes made:

- Replaced unserialize() with json_decode()
- Validated input structure before object creation
- Removed trust in user-supplied objects

This prevents object injection.

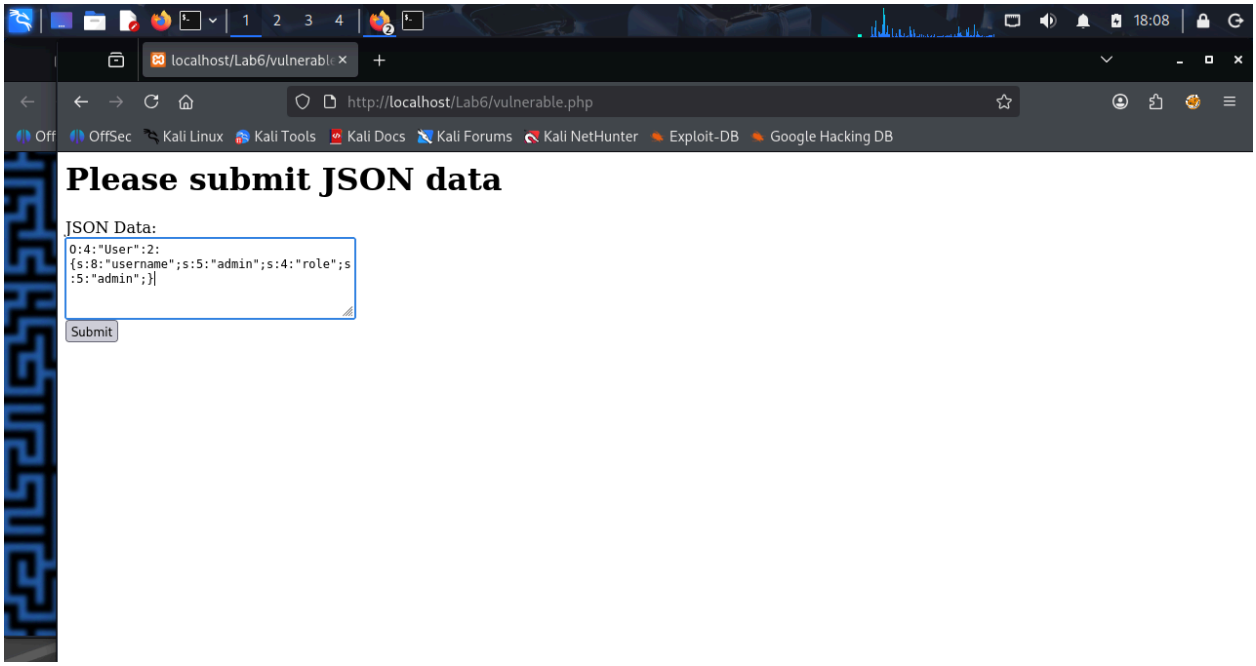


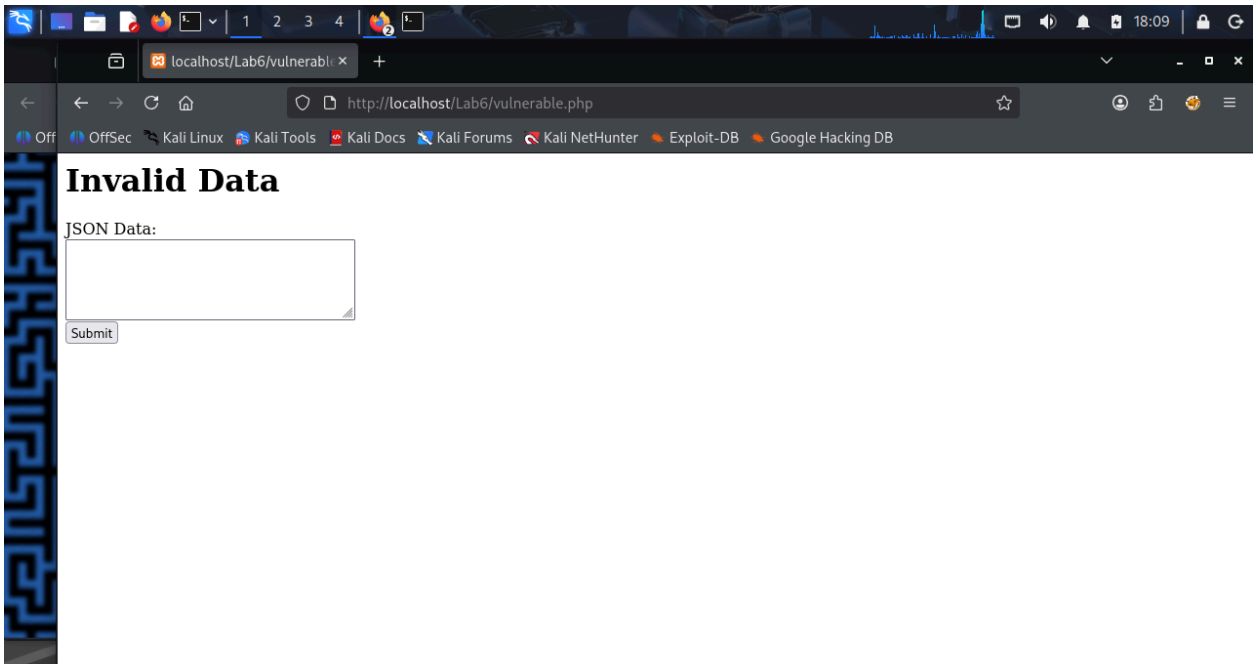
Testing the Fixed Application

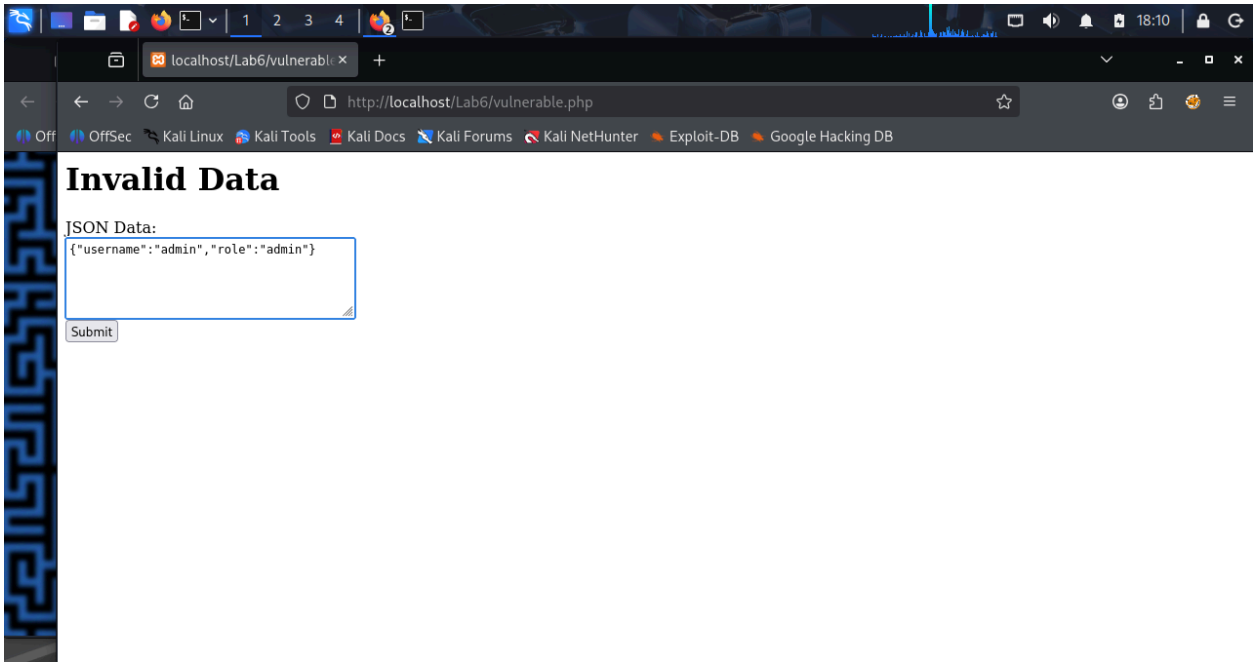
- Old serialized payload was rejected.
- Valid JSON data was accepted.

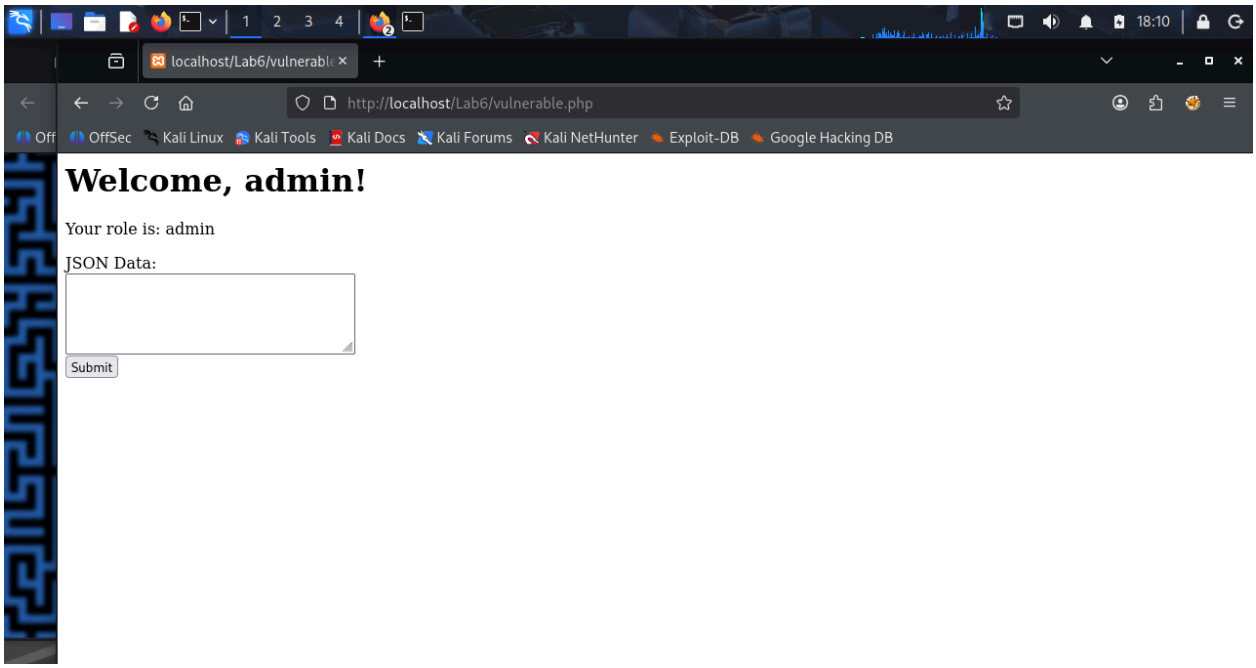
Valid JSON used:

{"username":"admin","role":"admin"}


18:08
localhost/Lab6/vulnerable
http://localhost/Lab6/vulnerable.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Please submit JSON data
JSON Data:
0:4:"User":2:
{s:8:"username";s:5:"admin";s:4:"role";s
:5:"admin";}
Submit


18:09
localhost/Lab6/vulnerable
http://localhost/Lab6/vulnerable.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Invalid Data
JSON Data:
Submit





Conclusion

Insecure deserialization allows attackers to inject malicious objects.
This can lead to privilege escalation or server crashes.

Using JSON and validating input prevents this vulnerability.
Applications must never unserialize untrusted data.